MANUAL DEL PROYECTO — AGROGESTIÓN

Índice

1. Introducción y contexto del proyecto
2. Objetivos del proyecto
3. Planificación y metodología
4. Primer trimestre: Análisis y diseño
    - 4.1. Estudio de viabilidad
    - 4.2. Análisis de requisitos
    - 4.3. Diseño de la base de datos
    - 4.4. Diseño de la arquitectura
    - 4.5. Diseño de la interfaz (mockups)
    - 4.6. Elección de tecnologías
5. Segundo trimestre: Desarrollo e implementación
    - 5.1. Configuración del entorno de desarrollo
    - 5.2. Configuración de Supabase
    - 5.3. Desarrollo del frontend
    - 5.4. Integración con Supabase
    - 5.5. Implementación de autenticación
    - 5.6. Implementación de roles y paneles
    - 5.7. Gestión de terrenos
    - 5.8. Gestión de tareas
    - 5.9. Relaciones gerente-capataz y capataz-trabajador
    - 5.10. Políticas de seguridad (RLS)
    - 5.11. Internacionalización (i18n)
    - 5.12. Comentarios en tareas
    - 5.13. Sistema de notificaciones
    - 5.14. Flujo de aceptación/rechazo de trabajadores
    - 5.15. Pruebas y corrección de errores
    - 5.16. Documentación
6. Cronograma del proyecto
7. Problemas encontrados y soluciones
8. Conclusiones
9. Agradecimientos

---

1. Introducción y contexto del proyecto

El presente documento recoge el proceso completo de desarrollo del proyecto **Agrogestión**, realizado como Proyecto de Fin de Ciclo del CFGS en Desarrollo de Aplicaciones Web (DAW) en el IES Albarregas de Mérida (España).

Agrogestión es una aplicación web que permite gestionar de forma digital el trabajo en explotaciones agrícolas. El proyecto nace de la necesidad real de organizar y coordinar las tareas entre los distintos perfiles que participan en la actividad agrícola: administradores, gerentes, capataces y trabajadores.

La aplicación se ha desarrollado a lo largo de dos trimestres académicos, abarcando desde el análisis inicial de requisitos hasta la implementación completa, las pruebas y la documentación final.

Datos del proyecto

Campo	Detalle
Nombre del proyecto	Agrogestión — Gestión agrícola web
Alumno	Juan Francisco Hurtado Pérez
Ciclo formativo	CFGS Desarrollo de Aplicaciones Web (DAW)
Centro educativo	IES Albarregas – Mérida (España)
Curso académico	2024/2025 – 2025/2026
Repositorio	https://github.com/Juanfrina/Agrogestion.Web
Tecnologías	React, TypeScript, Vite, TailwindCSS, Supabase

---

2. Objetivos del proyecto

Objetivos generales

- Desarrollar una aplicación web funcional que permita gestionar el trabajo agrícola de forma digital.
- Aplicar los conocimientos adquiridos durante el ciclo formativo en un proyecto completo y profesional.
- Utilizar tecnologías modernas del ecosistema web actual.

Objetivos específicos

ID	Objetivo
O1	Implementar un sistema de autenticación seguro con registro y login.
O2	Diseñar e implementar un modelo de datos relacional para la gestión agrícola.
O3	Crear una interfaz web responsive, accesible desde cualquier dispositivo.
O4	Desarrollar un sistema de roles con permisos diferenciados (Admin, Gerente, Capataz, Trabajador).
O5	Permitir la gestión de terrenos agrícolas (alta, baja lógica, edición).
O6	Implementar la creación, asignación y seguimiento de tareas agrícolas.
O7	Establecer relaciones entre gerentes-capataces y capataces-trabajadores.
O8	Aplicar políticas de seguridad RLS para proteger los datos según el rol.
O9	Documentar el proyecto de forma completa (usuario, técnico, despliegue).
O10	Desplegar la aplicación en un entorno accesible desde Internet.
O11	Implementar internacionalización (i18n) con soporte para español, inglés y rumano.
O12	Desarrollar un sistema de comentarios en tareas y notificaciones internas.
O13	Implementar un flujo de aceptación/rechazo de tareas por parte de los trabajadores.

---

3. Planificación y metodología

Metodología de trabajo

Se ha utilizado una metodología **iterativa e incremental**, combinando elementos de:

- **Desarrollo ágil:** Trabajo en iteraciones cortas con entregas parciales funcionales.
- **Modelo en cascada (adaptado):** El primer trimestre se centró en análisis y diseño, y el segundo en desarrollo e implementación.

Herramientas de planificación

Herramienta	Uso
GitHub	Control de versiones y almacenamiento del código fuente.
Visual Studio Code	IDE principal para desarrollo.
Supabase Dashboard	Gestión de la base de datos y autenticación.
Navegador (DevTools)	Depuración y pruebas de la interfaz.

División del trabajo

Período	Actividades principales
Primer trimestre	Análisis, diseño de la base de datos, arquitectura, mockups, elección de tecnologías.
Segundo trimestre	Configuración del entorno, desarrollo del frontend, integración con backend, pruebas, documentación.

---

4. Primer trimestre: Análisis y diseño

4.1. Estudio de viabilidad

Antes de comenzar el desarrollo, se realizó un estudio de viabilidad para valorar la factibilidad del proyecto:

Viabilidad técnica:

- Las tecnologías elegidas (React, Supabase) son maduras, bien documentadas y con gran comunidad.
- El alumno contaba con conocimientos previos de HTML, CSS, JavaScript y bases de datos SQL adquiridos durante el ciclo.
- Supabase como BaaS reduce significativamente la complejidad del backend.

Viabilidad económica:

- Todas las tecnologías utilizadas son **gratuitas** o tienen planes free suficientes para el proyecto.
- Supabase ofrece un plan gratuito con base de datos de 500 MB, autenticación ilimitada y 50.000 filas.
- Las plataformas de despliegue (Vercel, Netlify) ofrecen planes gratuitos para proyectos personales.

Viabilidad temporal:

- El proyecto se planificó para completarse en dos trimestres académicos, lo cual es suficiente para su alcance.

Conclusión: El proyecto es viable técnica, económica y temporalmente.

4.2. Análisis de requisitos

Se identificaron los requisitos funcionales y no funcionales del sistema (detallados en el Manual Técnico):

- **28 requisitos funcionales** que cubren: autenticación, gestión de usuarios, terrenos, tareas, relaciones entre roles, filtrado de datos, comentarios en tareas, notificaciones y flujo de aceptación/rechazo de trabajadores.
- **9 requisitos no funcionales** que cubren: usabilidad, rendimiento, seguridad, mantenibilidad, compatibilidad y disponibilidad.

Se identificaron **5 actores** del sistema: Visitante, Admin, Gerente, Capataz y Trabajador.

Se definieron **26 casos de uso** que describen las interacciones de cada actor con el sistema.

4.3. Diseño de la base de datos

Se diseñó un modelo relacional compuesto por **10 tablas**:

Tabla	Tipo
rol	Catálogo de roles del sistema (ADMIN, GERENTE, CAPATAZ, TRABAJADOR).
perfiles	Datos de perfil de cada usuario, vinculada a `auth.users`.
terreno	Terrenos agrícolas gestionados por gerentes.
estados_tarea	Catálogo de 6 estados (PENDIENTE, ASIGNADA, ACEPTADA, RECHAZADA, EN_PROGRESO, COMPLETADA).
tarea	Tareas agrícolas vinculadas a terrenos y usuarios.
gerente_capataz	Relación N:M entre gerentes y capataces con fechas de asignación/baja.
tarea_trabajador	Relación N:M entre tareas y trabajadores con estado de aceptación.
capataz_trabajador	Relación estable N:M entre capataces y trabajadores.
comentarios_tarea	Comentarios asociados a tareas por cualquier usuario participante.
notificaciones	Sistema de notificaciones internas (asignaciones, cambios de estado).

Decisiones de diseño:

- Se utilizó **UUID** como clave primaria para la tabla `perfiles`, vinculándola directamente con `auth.users` de Supabase mediante `ON DELETE CASCADE`.
- Se implementó **borrado lógico** (campo `fecha_baja`) en lugar de borrado físico para terrenos, usuarios y relaciones.
- Las relaciones N:M se resolvieron mediante tablas intermedias con claves primarias compuestas.
- Se normalizaron los estados de tarea en una tabla separada con 6 valores que soportan el flujo completo (incluyendo aceptación/rechazo de trabajadores).
- Se añadieron **triggers** automáticos para generar notificaciones al asignar tareas a capataces y trabajadores.

4.4. Diseño de la arquitectura

Se eligió una arquitectura **SPA + BaaS (Backend as a Service)**:

```
┌──────────────────┐          HTTPS          ┌──────────────────────┐
│   FRONTEND       │ ◄──────────────────► │   BACKEND            │
│   React + Vite   │                         │   Supabase           │
│   TailwindCSS    │                         │   - PostgreSQL       │
│   TypeScript     │                         │   - Auth             │
```

```
│                 │                   │   - API REST        │
│   (Navegador)   │                   │   - RLS             │
└─────────────────┘                   └─────────────────────┘
```

Justificación:

- React permite crear interfaces dinámicas y reactivas.
- Supabase elimina la necesidad de desarrollar un backend propio, reduciendo el tiempo de desarrollo.
- TailwindCSS permite un diseño responsive rápido y consistente.
- TypeScript añade seguridad de tipos y reduce errores en tiempo de desarrollo.

4.5. Diseño de la interfaz (mockups)

Se diseñaron wireframes para las pantallas principales:

Pantallas diseñadas:

Pantalla	Descripción
Landing Page	Página de bienvenida con acceso a login y registro.
Login	Formulario de inicio de sesión.
Registro	Formulario de registro de nuevo usuario.
Panel del Administrador	Tabla de usuarios con opciones de gestión.
Panel del Gerente	Secciones de terrenos, tareas y capataces.
Panel del Capataz	Lista de tareas asignadas con opciones de gestión.
Panel del Trabajador	Lista de tareas en las que participa.

Principios de diseño aplicados:

- Diseño limpio y minimalista.
- Colores verdes y terrosos evocando el ámbito agrícola.
- Interfaz responsive con tres breakpoints: móvil, tablet y escritorio.
- Navegación intuitiva con menú lateral en escritorio y menú hamburguesa en móvil.

4.6. Elección de tecnologías

Se evaluaron varias alternativas antes de elegir las tecnologías definitivas:

Frontend:

Opción evaluada	Decisión	Motivo
React	✅ Elegido	Gran ecosistema, amplia documentación, experiencia previa.
Vue.js	❌	Menor experiencia del alumno, ecosistema más reducido.
Angular	❌	Demasiado complejo para el alcance del proyecto.

Backend:

Opción evaluada	Decisión	Motivo

Supabase	✅ Elegido	BaaS completo, ahorra desarrollo de backend, incluye Auth.
Firebase	❌	Base de datos NoSQL, menos adecuada para relaciones complejas.
Node.js + Express	❌	Requiere más tiempo de desarrollo para API, auth, etc.

CSS:

Opción evaluada	Decisión	Motivo
TailwindCSS	✅ Elegido	Desarrollo rápido, consistente, responsive nativo.
Bootstrap	❌	Diseño más genérico, menos personalizable.
CSS puro	❌	Más lento y propenso a inconsistencias.

Lenguaje:

Opción evaluada	Decisión	Motivo
TypeScript	✅ Elegido	Tipado estático, menos errores, mejor autocompletado.
JavaScript	❌	Más propenso a errores por falta de tipado.

---

5. Segundo trimestre: Desarrollo e implementación

5.1. Configuración del entorno de desarrollo

Pasos realizados:

1. Instalación de Node.js (v20 LTS) y npm.
2. Creación del proyecto con `npm create vite@latest agrogestion -- --template react-ts`.
3. Instalación de Visual Studio Code con extensiones:
   - ES7+ React/Redux/React-Native snippets
   - Tailwind CSS IntelliSense
   - ESLint
   - Prettier
4. Inicialización del repositorio Git y conexión con GitHub.
5. Configuración de ESLint y TypeScript.

5.2. Configuración de Supabase

Pasos realizados:

1. Creación de la cuenta y proyecto en Supabase.
2. Ejecución del script `schema.sql` para crear las 10 tablas del modelo de datos, funciones, triggers y políticas RLS.
3. Inserción de datos iniciales: roles (ADMIN, GERENTE, CAPATAZ, TRABAJADOR) y estados de tarea (PENDIENTE, ASIGNADA, ACEPTADA, RECHAZADA, EN_PROGRESO, COMPLETADA).
4. Configuración del cliente Supabase en `src/database/supabase/Client.ts`.
5. Creación del archivo `.env` con las credenciales.
6. Ejecución de `seed_admin.sql` para crear el usuario administrador.
7. Verificación de la conexión desde el frontend.

5.3. Desarrollo del frontend

Estructura de carpetas creada:

```
src/
├── @types/         → Declaraciones de tipos globales (i18next.d.ts)
├── assets/         → Recursos estáticos (imágenes, iconos, fuentes, etc.)
│
├── components/     → Componentes reutilizables organizados por dominio
│   ├── admin/      → Componentes del panel de administración
│   ├── capataz/    → Componentes del panel de capataz
│   ├── cards/      → Componentes de tarjetas
│   ├── charts/     → Componentes de gráficos y visualización de datos
│   ├── common/     → Componentes comunes (Header, Footer, Layout, Sidebar, etc.)
│   ├── forms/      → Componentes de formularios (Login, Registro, Reset, etc.)
│   ├── gerente/    → Componentes del panel de gerente
│   ├── home/       → Componente Landing
│   ├── profile/    → Componente de perfil de usuario
│   ├── trabajador/ → Componentes del panel de trabajador
│   └── ui/         → Componentes UI genéricos (Table, Badge, SearchBar, Button, Select)
├── context/        → Contextos React (AuthContext, ThemeContext)
├── database/       → Capa de acceso a datos
│   ├── repositories/ → Interfaces de repositorios (patrón Repository)
│   └── supabase/     → Cliente Supabase + implementaciones de repositorios
│       └── RPCs/     → Funciones RPC de Supabase
├── hooks/          → Custom hooks (useAuth, useTheme)
├── interfaces/     → Tipos TypeScript para las entidades del sistema
├── lib/            → Utilidades y configuración (constants, i18n)
├── locales/        → Archivos de traducción (es.json, en.json, ro.json)
├── pages/          → Vistas/páginas de la aplicación
├── router/         → Configuración de rutas (AppRouter, ProtectedRoute, PublicRoute)
├── store/          → Gestión de estado global (authStore)
├── styles/         → Hojas de estilos (App.css, index.css)
└── utils/          → Funciones auxiliares (dates, regex, validators)
```

Componentes desarrollados:

Componente	Carpeta	Descripción
Layout	common	Estructura base con header, sidebar y área de contenido.
Header	common	Barra superior con logo, usuario, idioma y cerrar sesión.
Footer	common	Pie de página con créditos.
Sidebar	common	Menú lateral con enlaces según el rol del usuario.
Modal	common	Ventana modal para formularios de creación/edición.
Spinner	common	Indicador de carga.
Alert	common	Componente de mensajes y notificaciones.
ThemeToggle	common	Selector de tema claro/oscuro.

TareaComentarios	common	Hilo de comentarios en una tarea con envío de mensajes.
LanguageSwitcher	common	Selector de idioma (español, inglés y rumano).
LoginForm	forms	Formulario de inicio de sesión.
RegistroForm	forms	Formulario de registro de nuevo usuario.
ResetPasswordForm	forms	Formulario de recuperación de contraseña.
InputField	forms	Campo de formulario con validación y estilos consistentes.
Button	ui	Botón estilizado con variantes (primario, secundario, peligro).
Table	ui	Tabla genérica reutilizable para mostrar datos.
SearchBar	ui	Barra de búsqueda con filtrado.
Select	ui	Selector desplegable estilizado.
Badge	ui	Etiqueta visual para estados y categorías.
Card	cards	Tarjeta de información reutilizable.
MensualChart	charts	Gráfico de barras con datos mensuales para visualización.
RoleDistributionChart	charts	Gráfico de distribución de usuarios por rol.
Landing	home	Contenido de la página de inicio.
Profile	profile	Vista del perfil de usuario.
AdminDashboard	admin	Panel principal del administrador.
UserTable	admin	Tabla de gestión de usuarios.
GerenteDashboard	gerente	Panel principal del gerente.
TerrenoList	gerente	Listado de terrenos del gerente.
TerrenoForm	gerente	Formulario de alta/edición de terrenos.
TareaGerenteList	gerente	Listado de tareas del gerente.
TareaForm	gerente	Formulario de creación de tareas.
MiEquipoGerente	gerente	Gestión del equipo de capataces del gerente.
CapatazDashboard	capataz	Panel principal del capataz.
TareaCapatazList	capataz	Listado de tareas asignadas al capataz.
TrabajadorAsignacion	capataz	Gestión de asignación de trabajadores a tareas.
MiEquipoCapataz	capataz	Gestión del equipo de trabajadores del capataz.
TrabajadorDashboard	trabajador	Panel principal del trabajador.
MisTareasList	trabajador	Listado de tareas del trabajador.

ProtectedRoute	router	Componente que protege rutas según autenticación y rol.
PublicRoute	router	Componente para rutas públicas (redirige si hay sesión).

5.4. Integración con Supabase

Se implementó una capa de acceso a datos siguiendo el **patrón Repository**, con interfaces abstractas y sus implementaciones concretas para Supabase:

Interfaces de repositorio (`database/repositories/`):

Repositorio	Operaciones
AuthRepository	signUp, signIn, signOut, resetPassword, onAuthStateChange
UsuarioRepository	getAll, getById, getByEmail, update, deactivate
TerrenoRepository	getAll, getByGerente, create, update, deactivate
TareaRepository	getAll, getByGerente, getByCapataz, create, updateEstado
AdminRepository	getAllUsers, updateUserRole, deactivateUser

Implementaciones Supabase (`database/supabase/`):

Implementación	Repositorio base
SupabaseAuthRepository	AuthRepository
SupabaseUsuarioRepository	UsuarioRepository
SupabaseTerrenoRepository	TerrenoRepository
SupabaseTareaRepository	TareaRepository
SupabaseAdminRepository	AdminRepository

Además, se crearon funciones RPC en `database/supabase/RPCs/` para operaciones especiales como verificar si un email ya está registrado (`isEmailTaken.ts`).

Ejemplo de repositorio (patrón):

```
import { supabase } from '../supabase/Client';

export const TerrenoRepository = {
    getByGerente: async (idGerente: string) => {
        const { data, error } = await supabase
            .from('terreno')
            .select('*')
            .eq('id_gestor', idGerente)
            .is('fecha_baja', null);
        if (error) throw error;
        return data;
    },
```

```
    // ... más métodos
};
```

5.5. Implementación de autenticación

Se integró Supabase Auth para el sistema de autenticación:

Funcionalidades implementadas:

Funcionalidad	Implementación
Registro	`supabase.auth.signUp()` + trigger crea fila en `perfiles`.
Login	`supabase.auth.signInWithPassword()`.
Logout	`supabase.auth.signOut()`.
Recuperar contraseña	`supabase.auth.resetPasswordForEmail()`.
Persistencia de sesión	Token JWT almacenado automáticamente por Supabase.
Contexto de auth	React Context + Provider para estado global de sesión.
Store de auth	Estado centralizado de autenticación (`authStore.ts` con Zustand).

5.6. Implementación de roles y paneles

Se creó un sistema de redirección basado en el rol del usuario:

Rol	Ruta	Componente
ADMIN	/admin	AdminPage.tsx
GERENTE	/gerente	GerentePage.tsx
CAPATAZ	/capataz	CapatazPage.tsx
TRABAJADOR	/trabajador	TrabajadorPage.tsx
Sin sesión	/	LandingPage.tsx
Sin sesión	/login	Login.tsx
Sin sesión	/registro	Registro.tsx
Sin sesión	/reset-password	ResetPasswordPage.tsx

Se implementó un componente `ProtectedRoute` que:

1. Verifica si el usuario está autenticado.
2. Consulta el rol del usuario en la tabla `perfiles`.
3. Redirige al panel correspondiente o al login si no tiene sesión.

También se implementó un componente `PublicRoute` que redirige a los usuarios ya autenticados a su panel correspondiente cuando intentan acceder a rutas públicas (login, registro).

La configuración de rutas se centraliza en `router/routes.ts` y se orquesta desde `router/AppRouter.tsx`.

5.7. Gestión de terrenos

Se implementó el CRUD completo de terrenos para el rol Gerente:

Operación	Descripción
Listar	Tabla con todos los terrenos activos del gerente.
Crear	Formulario modal para dar de alta un nuevo terreno.
Editar	Formulario modal precargado con los datos del terreno.
Baja lógica	Botón para desactivar un terreno (campo `fecha_baja`).

5.8. Gestión de tareas

Se implementó la gestión de tareas con los siguientes flujos:

Desde el panel del Gerente:

- Crear nueva tarea seleccionando terreno y capataz.
- Ver listado de todas las tareas con filtros por estado y terreno.
- Consultar el progreso de las tareas.

Desde el panel del Capataz:

- Ver tareas asignadas con su estado actual.
- Tomar una tarea pendiente (cambia a "En progreso").
- Completar una tarea (cambia a "Completada").
- Asignar trabajadores de apoyo a una tarea.

Desde el panel del Trabajador:

- Ver las tareas en las que participa.
- Consultar detalles de cada tarea.

5.9. Relaciones gerente-capataz y capataz-trabajador

Se implementaron las dos relaciones N:M del modelo:

Gerente → Capataz (tabla `gerente_capataz`):

- El gerente puede asociar capataces a su equipo.
- Al crear una tarea, el gerente selecciona un capataz de su equipo.

Capataz → Trabajador (tabla `capataz_trabajador`):

- El capataz gestiona su equipo de trabajadores habituales.
- Al asignar apoyo a una tarea, selecciona entre sus trabajadores (tabla `tarea_trabajador`).

5.10. Políticas de seguridad (RLS)

Se configuraron políticas de Row Level Security en Supabase para cada tabla:

Tabla	Políticas principales
perfiles	Cada usuario ve su perfil; admin ve y edita todos.

terreno	Solo el gerente propietario puede ver, crear y editar.
tarea	Gerente ve las que creó; capataz ve las asignadas.
gerente_capataz	Acceso según pertenencia a la relación.
tarea_trabajador	Acceso según participación en la tarea.
capataz_trabajador	Acceso según pertenencia a la relación.
comentarios_tarea	Lectura/escritura para participantes de la tarea.
notificaciones	Solo el destinatario puede ver y marcar como leída.
rol	Lectura para todos los autenticados.
estados_tarea	Lectura para todos los autenticados.

Se configuraron aproximadamente **50 políticas RLS** en total, junto con **8 funciones auxiliares** (como `get_my_role()`, `is_admin()`, `tarea_ids_by_gerente()`, etc.) para evitar recursión en las políticas y optimizar las consultas de permisos.

5.11. Internacionalización (i18n)

Se implementó soporte multiidioma utilizando **i18next** y **react-i18next**:

Aspecto	Detalle
Librería	i18next 25.x + react-i18next 16.x
Idiomas soportados	Español (es), Inglés (en) y Rumano (ro).
Archivos de traducción	`src/locales/es.json`, `en.json`, `ro.json`.
Detección automática	Se detecta el idioma del navegador del usuario.
Persistencia	El idioma seleccionado se guarda en `localStorage`.
Componente	`LanguageSwitcher` permite cambiar de idioma en cualquier momento.
Cobertura	Toda la interfaz está traducida: menús, formularios, mensajes, botones, etc.
Tipado	Declaración de tipos en `@types/i18next.d.ts` para autocompletado.

5.12. Comentarios en tareas

Se implementó un sistema de comentarios dentro de cada tarea:

Aspecto	Detalle
Tabla	`comentarios_tarea` con FK a `tarea` (ON DELETE CASCADE) y `perfiles`.
Componente	`TareaComentarios` muestra el hilo de comentarios y permite enviar nuevos.
Permisos	Pueden comentar todos los participantes de una tarea (gerente, capataz, trabajadores asignados).

Visualización	Los comentarios se muestran ordenados cronológicamente con el nombre del autor y la fecha.

5.13. Sistema de notificaciones

Se implementó un sistema de notificaciones internas automáticas:

Aspecto	Detalle
Tabla	`notificaciones` con campos: tipo, título, mensaje, id_tarea, leída.
Triggers	`trg_notif_tarea_asignada_capataz` (notifica al capataz cuando se le asigna tarea) y `trg_notif_trabajador_asignado` (notifica al trabajador cuando se le asigna a una tarea).
Tipos de notificación	10 tipos: TAREA_ASIGNADA, TAREA_ACEPTADA, TAREA_RECHAZADA, TAREA_EN_PROGRESO, TAREA_COMPLETADA, TRABAJADOR_ASIGNADO, TRABAJADOR_ACEPTO, TRABAJADOR_RECHAZO, COMENTARIO_NUEVO, ASOCIACION_NUEVA.
Lectura	El destinatario puede marcar notificaciones como leídas.
RLS	Solo el destinatario puede ver sus propias notificaciones.

5.14. Flujo de aceptación/rechazo de trabajadores

Se implementó un flujo completo para que los trabajadores puedan aceptar o rechazar las tareas que les asignan:

Aspecto	Detalle
Tabla	`tarea_trabajador` con campo `estado` (PENDIENTE, ACEPTADA, RECHAZADA, COMPLETADA).
Asignación	El capataz asigna trabajadores; el estado inicial es PENDIENTE.
Aceptación/Rechazo	El trabajador puede aceptar o rechazar desde su panel.
Registro	Se guardan `fecha_asignacion` y `fecha_respuesta`.
Notificación	Al asignar un trabajador se genera automáticamente una notificación mediante trigger.

5.15. Pruebas y corrección de errores

Se realizaron tres tipos de pruebas:

1. **Pruebas funcionales (22 tests):** Verificación de cada caso de uso implementado, incluyendo comentarios, notificaciones y flujo de aceptación/rechazo.
2. **Pruebas de interfaz (5 tests):** Comprobación del diseño responsive, cambio de idioma y menú hamburguesa en distintos dispositivos y navegadores.
3. **Pruebas de seguridad (4 tests):** Verificación de las políticas RLS, protección de rutas y aislamiento de datos entre roles.

Principales errores detectados y corregidos:

Error	Solución aplicada
Token JWT concatenado con URL en .env	Se separaron correctamente las variables .env.

TailwindCSS no se cargaba	Se añadió el plugin `@tailwindcss/vite` en `vite.config.ts`.
Redirección incorrecta tras login	Se corrigió la lógica de consulta de rol del usuario.
Error de permisos al crear terreno	Se ajustaron las políticas RLS de la tabla `terreno`.
Formularios no validaban campos vacíos	Se añadió validación en el frontend antes del envío.
Erratas en nombres de archivos	Se corrigieron 7 nombres (LoginFrom→LoginForm, etc.).
Archivos de estructura faltantes	Se crearon 37 archivos vacíos de scaffolding.
Recursión infinita en políticas RLS	Se crearon funciones auxiliares (`get_my_role`, `is_admin`, etc.) con `SECURITY DEFINER` para evitar consultas recursivas.
Conflicto de políticas en tarea_trabajador	Se reescribieron las políticas RLS de la tabla con script `fix_tarea_trabajador.sql`.

5.16. Documentación

En la fase final del proyecto se elaboraron cuatro documentos:

Documento	Contenido
Manual de Usuario	Guía para que cualquier usuario pueda usar la aplicación.
Manual Técnico	Arquitectura, tecnologías, base de datos, requisitos, pruebas.
Manual de Despliegue	Instrucciones completas de instalación y puesta en marcha.
Manual del Proyecto	Memoria del trabajo realizado durante los dos trimestres.

---

6. Cronograma del proyecto

Primer trimestre (Octubre – Diciembre)

Semana	Actividad
Semana 1	Definición del proyecto, estudio de viabilidad.
Semana 2	Análisis de requisitos funcionales y no funcionales.
Semana 3	Diseño del modelo de datos (esquema E-R, diccionario).
Semana 4	Diseño de la arquitectura (SPA + BaaS).
Semana 5	Evaluación y selección de tecnologías.
Semana 6	Diseño de mockups e interfaz de usuario.

Semana 7	Definición de casos de uso y diagramas.
Semana 8	Creación del repositorio y scaffolding del proyecto.
Semana 9	Configuración inicial de Supabase (tablas, datos, políticas).
Semana 10	Redacción de documentación preliminar de análisis y diseño.

Segundo trimestre (Enero – Marzo)

Semana	Actividad
Semana 11	Configuración del entorno: Vite, React, TypeScript, Tailwind.
Semana 12	Desarrollo: Landing Page, Login, Registro.
Semana 13	Desarrollo: Integración con Supabase Auth (login/registro).
Semana 14	Desarrollo: Sistema de roles, contexto de autenticación.
Semana 15	Desarrollo: Panel del Administrador (gestión de usuarios).
Semana 16	Desarrollo: Panel del Gerente (terrenos).
Semana 17	Desarrollo: Panel del Gerente (tareas, asignación capataces).
Semana 18	Desarrollo: Panel del Capataz (tareas, cambio de estado).
Semana 19	Desarrollo: Panel del Capataz (trabajadores), Panel Trabajador.
Semana 20	Configuración de políticas RLS en Supabase (~50 políticas).
Semana 21	Internacionalización (i18n): español, inglés y rumano.
Semana 22	Comentarios en tareas y sistema de notificaciones.
Semana 23	Flujo de aceptación/rechazo de trabajadores.
Semana 24	Pruebas funcionales, de interfaz y de seguridad.
Semana 25	Corrección de errores y ajustes finales.
Semana 26	Despliegue en producción.
Semana 27	Redacción de documentación final (4 manuales).

---

7. Problemas encontrados y soluciones

7.1. Problemas técnicos

Problema	Solución
TailwindCSS v4 no se integraba con Vite 7	Se investigó la documentación actualizada y se configuró el plugin `@tailwindcss/vite` correctamente.

Supabase RLS bloqueaba consultas legítimas	Se revisaron y refinaron las políticas de seguridad paso a paso, probando cada una individualmente.
Error de CORS al conectar desde localhost	Se verificó que Supabase permite conexiones desde localhost por defecto con la anon key.
Manejo de sesiones al recargar la página	Se implementó `supabase.auth.onAuthStateChange()` para persistir el estado de autenticación.
TypeScript reportaba errores de tipo con datos de Supabase	Se crearon interfaces TypeScript para las tablas de la base de datos (carpeta `interfaces/`).
Hot Module Replacement lento en Windows	Se excluyó la carpeta `node_modules` del antivirus de Windows.

7.2. Problemas de planificación

Problema	Solución
Subestimación del tiempo para políticas RLS	Se dedicó una semana completa adicional a la configuración de seguridad.
Complejidad de las relaciones N:M	Se simplificó la interfaz para gestionar las relaciones de forma intuitiva.
Documentación consumió más tiempo del previsto	Se comenzó la documentación en paralelo con las últimas fases de desarrollo.

---

8. Conclusiones

Cumplimiento de objetivos

Objetivo	Descripción	Estado
O1	Sistema de autenticación con registro y login	✅ Cumplido
O2	Modelo de datos relacional	✅ Cumplido
O3	Interfaz responsive	✅ Cumplido
O4	Sistema de roles con permisos diferenciados	✅ Cumplido
O5	Gestión de terrenos	✅ Cumplido
O6	Gestión de tareas	✅ Cumplido
O7	Relaciones gerente-capataz y capataz-trabajador	✅ Cumplido
O8	Políticas de seguridad RLS	✅ Cumplido
O9	Documentación completa	✅ Cumplido
O10	Despliegue accesible desde Internet	✅ Cumplido
O11	Internacionalización (i18n) en 3 idiomas	✅ Cumplido

O12	Comentarios en tareas y notificaciones	✅ Cumplido
O13	Flujo de aceptación/rechazo de trabajadores	✅ Cumplido

Valoración personal

El desarrollo de Agrogestión ha sido una experiencia muy formativa que ha permitido:

- **Aplicar conocimientos teóricos** adquiridos durante el ciclo en un proyecto real y completo.
- **Aprender tecnologías modernas** como React con TypeScript, Vite, TailwindCSS y Supabase, que son ampliamente utilizadas en la industria.
- **Afrontar problemas reales** de desarrollo (configuración de entornos, seguridad, diseño de base de datos) y encontrar soluciones.
- **Mejorar la capacidad de planificación** y gestión del tiempo en un proyecto de varios meses.
- **Desarrollar habilidades de documentación** técnica, esenciales en el ámbito profesional.

El proyecto cumple todos los objetivos planteados inicialmente y sienta las bases para futuras ampliaciones como geolocalización de terrenos, informes en PDF, notificaciones push en tiempo real o una versión de aplicación móvil.

Conocimientos aplicados del ciclo DAW

Módulo del ciclo	Aplicación en el proyecto
Lenguajes de marcas (HTML/CSS)	Estructura y estilos de la interfaz web.
Programación (JavaScript/TypeScript)	Lógica del frontend y comunicación con Supabase.
Bases de datos (SQL)	Diseño del modelo relacional en PostgreSQL.
Desarrollo web en entorno cliente	React, manejo de estado, componentes, routing.
Desarrollo web en entorno servidor	API REST de Supabase, autenticación, RLS.
Despliegue de aplicaciones web	Configuración de Vite, build, despliegue.
Diseño de interfaces web	UX/UI con TailwindCSS, responsive design.
Empresa e iniciativa emprendedora	Planificación, documentación y gestión del proyecto.

---

9. Agradecimientos

Quiero agradecer:

- A los **profesores del IES Albarregas** por la formación recibida durante estos dos años del ciclo formativo, especialmente por su apoyo y orientación durante el desarrollo de este proyecto.
- A la **comunidad open source** que mantiene las herramientas y librerías utilizadas (React, Vite, Supabase, TailwindCSS), facilitando el desarrollo de aplicaciones modernas.
- A mi **familia y compañeros** por su apoyo y paciencia durante el proceso.